hetmacro Codebook

A Comprehensive Guide to Numerical Tools
for Heterogeneous Agent Macroeconomics

Rafael Lincoln
New York University

January 2026

Contents

CHAPTER 1

Introduction

1.1 Purpose and Philosophy

The `hetmacro` package provides a self-contained, modular toolkit for solving structural macroeconomic models with heterogeneity. These models—including Aiyagari, Krusell-Smith, and HANK models—require agents to make optimal decisions over continuous state spaces, typically involving savings/consumption choices under idiosyncratic and aggregate uncertainty.

Solving such models numerically requires a carefully orchestrated pipeline:

1. **Discretization**: Continuous state spaces must be approximated by grids; continuous stochastic processes (like income) must be discretized into Markov chains.

2. **Backward Iteration**: Given prices, solve for optimal policy functions via dynamic programming (VFI, EGM).

3. **Forward Iteration**: Given policies, compute the stationary distribution of agents across states.

4. **Equilibrium**: Find prices that clear markets (e.g., asset supply equals capital demand).

5. **Linearization**: For aggregate dynamics, compute sequence-space Jacobians.

This codebook documents each module in `hetmacro`, explaining not just *what* each function does, but *why* it exists and *how* it connects to other tools in the pipeline.

1.2 Package Architecture

The package is organized into two layers:

1.2.1 Foundational Modules

General-purpose numerical tools that can be used for any computational economics application:

- `grids.py` – State space discretization
- `quadrature.py` – Numerical integration
- `interpolation.py` – Function approximation
- `optimize.py` – Root-finding and optimization
- `markov.py` – Markov chain tools
- `utils.py` – Economic primitives and utilities

1.2.2 Heterogeneous Agent Tools

Specialized routines for HA models:

- `backward.py` – Dynamic programming solvers
- `forward.py` – Distribution iteration
- `steady_state.py` – Equilibrium solvers
- `ssj.py` – Sequence-space Jacobians
- `dp_tools.py`, `distributions.py` – High-level wrappers

CHAPTER 2

Foundational Modules

2.1 `grids.py` – State Space Discretization

2.1.1 Introduction and Economic Motivation

In heterogeneous agent models, agents make decisions over continuous state variables—typically assets $a \in [\underline{a}, \bar{a}]$ and possibly other states like productivity, human capital, or housing. Since computers cannot store functions over continuous domains, we must discretize these spaces into finite grids.

The choice of grid matters substantially for accuracy and computational cost:

- **Near borrowing constraints**: Policy functions often exhibit kinks near $a = \underline{a}$ where agents are constrained. Dense grid coverage here improves accuracy.
- **In the tail**: Wealthy agents' behavior matters less for aggregates but requires coverage for numerical stability.
- **Around specific values**: Some models have kinks at particular thresholds (e.g., tax brackets, collateral constraints).

This module provides flexible grid construction supporting various spacing strategies and the ability to concentrate points around specified locations.

2.1.2 Core Class: `GridSpec`

```python
@dataclass
class GridSpec:
    lower: float           # Lower bound of grid
    upper: float           # Upper bound of grid
    n_points: int          # Number of grid points
    spacing: str = "linear"          # Spacing method
    power: float = 1.0               # Exponent for power spacing
    concentration_points: Optional[Sequence[float]] = None
    concentration_weight: float = 0.0
    concentration_radius: Optional[float] = None
    custom_func: Optional[Callable] = None
```

Purpose.
A specification object for one dimension of a grid. Use `GridSpec` when constructing multi-dimensional grids via `make_grid_nd`.

Arguments.

`lower`
Lower bound of the state variable (e.g., borrowing limit $\underline{a}$).

`upper`
Upper bound (e.g., maximum asset level $\bar{a}$).

`n_points`
Number of grid points. More points $\Rightarrow$ higher accuracy but slower computation.

`spacing`
One of `"linear"`, `"power"`, `"log"`, `"double_exp"`, `"chebyshev"`, or `"custom"`.

`power`
For `spacing="power"`: exponent γ where grid is $\underline{a} + (\bar{a} - \underline{a}) \cdot t^{\gamma}$ for $t \in [0, 1]$. Values > 1 concentrate points near lower bound.

`concentration_points`
Sequence of values around which to concentrate grid points (e.g., kink locations).

`concentration_weight`
Strength $\in [0, 1]$ of concentration effect.

`concentration_radius`
Scale parameter for concentration kernel. Defaults to 10% of range.

`custom_func`
For `spacing="custom"`: a function mapping $[0, 1] \to [\text{lower}, \text{upper}]$.

2.1.3 Function: `make_grid_1d`

```python
def make_grid_1d(
    lower: float,
    upper: float,
    n: int,
    spacing: str = "linear",
    power: float = 1.0,
    concentration_points: Optional[Sequence[float]] = None,
    concentration_weight: float = 0.0,
    concentration_radius: Optional[float] = None,
    custom_func: Optional[Callable] = None,
) -> np.ndarray
```

Purpose.
Create a one-dimensional grid with flexible spacing. This is the workhorse function for discretizing a single state variable.

Arguments.

`lower, upper`
Bounds of the grid (inclusive). For assets, typically `lower=0` (or natural borrowing limit) and `upper` large enough to be non-binding.

`n`
Number of grid points (must be ≥ 2).

`spacing`
Grid spacing method:

- `"linear"`: Equidistant points. Simple but inefficient near constraints.
- `"power"`: $a_i = \underline{a} + (\bar{a} - \underline{a})(i/(n-1))^{\gamma}$. Use $\gamma > 1$ to concentrate near $\underline{a}$.
- `"log"`: Logarithmically spaced. Requires `lower > 0`.

- `"double_exp"`: Rognlie-style double exponential. Very dense near lower bound.
- `"chebyshev"`: Chebyshev nodes. Optimal for polynomial approximation.
- `"custom"`: User-provided transformation.

`power`
Exponent for power spacing (default 1.0 = linear).

`concentration_*`
Optional point concentration (see `GridSpec`).

`custom_func`
Required if `spacing="custom"`.

Returns. 1D NumPy array of grid points, sorted and with exact endpoints.

Connections. Output feeds into `backward.py` for policy iteration, `interpolation.py` for function evaluation, and `forward.py` for distribution iteration.

Example.

```python
import numpy as np
from hetmacro.grids import make_grid_1d

# Asset grid with power spacing (dense near borrowing constraint)
a_grid = make_grid_1d(
    lower=0.0,
    upper=50.0,
    n=200,
    spacing="power",
    power=2.0
)
print(f"First 5 points: {a_grid[:5]}")
# Dense near 0, sparse near 50

# Grid with concentration around kink at a=10
a_grid_kink = make_grid_1d(
    lower=0.0,
    upper=50.0,
    n=200,
```

```
    spacing="linear",
    concentration_points=[10.0],
    concentration_weight=0.5,
    concentration_radius=2.0
)
```

2.1.4 Function: `make_grid_nd`

```
def make_grid_nd(
    specs: Sequence[GridSpec],
    cartesian: bool = False,
    tensor: bool = False,
    order: str = "C",
) -> Tuple[List[np.ndarray], Optional[np.ndarray]]
```

Purpose.
Construct multi-dimensional grids from a list of `GridSpec` objects. Essential for models with multiple state variables (e.g., assets and human capital, or assets and housing).

Arguments.

`specs`
Sequence of `GridSpec` objects, one per dimension.

`cartesian`
If `True`, also return the Cartesian product of all grids as a 2D array where each row is a state combination.

`tensor`
If `True`, return a tensor grid of shape $(n_1, n_2, \ldots, n_d, d)$.

`order`
Memory layout: `"C"` (row-major) or `"F"` (column-major, Fortran-style).

Returns. Tuple of (list of 1D grids, optional Cartesian/tensor grid).

Connections. The 1D grids are used independently in `interpolation.py`; the Cartesian product is useful for computing expectations over joint distributions; the tensor grid is useful for reshaping policies over full state tensors.

Example.

```python
from hetmacro.grids import GridSpec, make_grid_nd

# Two-dimensional grid: assets and human capital
specs = [
    GridSpec(lower=0, upper=50, n_points=100, spacing="power", power=2),
    GridSpec(lower=0.5, upper=2.0, n_points=20, spacing="linear"),
]
grids, product = make_grid_nd(specs, cartesian=True)
a_grid, h_grid = grids
print(f"Asset grid: {len(a_grid)} points")
print(f"Human capital grid: {len(h_grid)} points")
print(f"State space: {product.shape[0]} total states")

# Tensor grid with shape (n_a, n_h, 2)
grids, tensor = make_grid_nd(specs, tensor=True)
print(tensor.shape)
```

2.1.5 Function: `make_asset_grid`

```python
def make_asset_grid(
    amin: float,
    amax: float,
    n: int,
    curvature: float = 2.0,
) -> np.ndarray
```

Purpose.

Convenience function for the common case of an asset grid with power spacing. This is a thin wrapper around `make_grid_1d` with sensible defaults for incomplete markets models.

Arguments.

`amin`

Borrowing limit (often 0 or natural borrowing limit).

`amax`
Upper bound on assets.

`n`
Number of grid points.

`curvature`
Power exponent (default 2.0 works well in practice).

Returns. 1D array of asset grid points.

Example.

```
from hetmacro.grids import make_asset_grid

# Standard Aiyagari-style asset grid
a_grid = make_asset_grid(amin=0, amax=50, n=500, curvature=2.0)
```

2.1.6 Function: `double_exponential_grid`

```
def double_exponential_grid(amin: float, amax: float, n: int) -> np.ndarray
```

Purpose.
Rognlie-style double exponential grid, which provides extremely dense coverage near the lower bound. Useful when policy functions have sharp features near constraints.

Mathematical form: Let $\bar{u} = \log(1 + \log(1 + a_{\max} - a_{\min}))$. The grid is:

$$a_i = a_{\min} + \exp(\exp(u_i) - 1) - 1, \quad u_i \in \text{linspace}(0, \bar{u}, n)$$

2.1.7 Function: `cartesian_product` / `gridmake`

```
def cartesian_product(*grids, order: str = "C") -> np.ndarray
def gridmake(*grids, order: str = "C") -> np.ndarray  # alias
```

Purpose.
Compute the Cartesian product of multiple 1D grids. Each row of the output represents one combination of state values.

Arguments.

`*grids`
Variable number of 1D arrays.

`order`
Memory layout for iteration order.

Returns. 2D array of shape $(n_1 \times n_2 \times \cdots, d)$ where d is the number of grids.

Example.

```
from hetmacro.grids import cartesian_product
import numpy as np

a_grid = np.array([0, 1, 2])
e_grid = np.array([0.5, 1.0, 1.5])
states = cartesian_product(a_grid, e_grid)
# states[0] = [0, 0.5], states[1] = [0, 1.0], etc.
```

2.1.8 Function: `tensor_product`

```
def tensor_product(*grids) -> np.ndarray
```

Purpose.
Construct a tensor grid that preserves the full $n_1 \times \cdots \times n_d$ structure, with the last axis storing coordinates.

Arguments.

`*grids`
Variable number of 1D arrays.

Returns. Array of shape $(n_1, n_2, \ldots, n_d, d)$.

Example.

```python
from hetmacro.grids import tensor_product
import numpy as np

a_grid = np.array([0, 1, 2])
e_grid = np.array([0.5, 1.0])
tensor = tensor_product(a_grid, e_grid)
# tensor.shape == (3, 2, 2)
```

2.1.9 Function: `smolyak_sparse_grid`

```python
def smolyak_sparse_grid(
    specs: Sequence[GridSpec],
    level: int,
    rule: str = "clenshaw_curtis",
    unique_decimals: int = 14,
) -> np.ndarray
```

Purpose.
Construct a Smolyak sparse grid on a hyper-rectangle using nested Clenshaw–Curtis nodes to reduce the curse of dimensionality.

Arguments.

`specs`
GridSpec list; uses only `lower` and `upper` per dimension.

`level`
Smolyak level (≥ 1). Higher levels add more points.

`rule`
1D nested rule (currently `"clenshaw_curtis"`).

`unique_decimals`
Decimal rounding for deduplication of nested nodes.

Returns. Array of unique sparse-grid points of shape (n_{points}, d).

Example.

```python
from hetmacro.grids import GridSpec, smolyak_sparse_grid

specs = [
    GridSpec(lower=-1, upper=1, n_points=0),
    GridSpec(lower=-1, upper=1, n_points=0),
]
points = smolyak_sparse_grid(specs, level=3)
print(points.shape)
```

2.2 `quadrature.py` – Numerical Integration

2.2.1 Introduction and Economic Motivation

Expectations are fundamental in dynamic economics. Agents form expectations over future states:

$$\mathbb{E}[V(a', e') \mid e] = \int V(a', e')\, dF(e' \mid e)$$

When shocks are continuous (e.g., normally distributed innovations), we cannot sum over states—we must integrate. Quadrature rules approximate integrals using weighted sums:

$$\int f(x)w(x)\, dx \approx \sum_{i=1}^{n} w_i f(x_i)$$

where $\{x_i\}$ are nodes and $\{w_i\}$ are weights chosen to make the approximation exact for polynomials up to some degree.

Key insight: Different distributions require different quadrature rules. Gauss-Hermite is optimal for normally distributed shocks; Gauss-Legendre for bounded uniform distributions.

Theory: Gaussian Quadrature

Gaussian quadrature is a family of methods for approximating integrals of the form:

$$\int_a^b f(x)\, w(x)\, dx \approx \sum_{i=1}^{n} w_i\, f(x_i)$$

where $w(x)$ is a **weighting function** (not to be confused with the quadrature weights w_i).

The key insight is that if the nodes $\{x_i\}$ are chosen as the roots of orthogonal polynomials associated with $w(x)$, then the approximation is **exact for polynomials of degree up to** $2n-1$. This is twice as accurate as methods with equally-spaced nodes (like Simpson or Trapezoidal rules).

Each quadrature rule in this module corresponds to a specific weighting function and its associated orthogonal polynomials:

Function	Weight function $w(x)$	Domain	Orthogonal Polynomial
`qnwlege`	1 (uniform)	$[a, b]$	Legendre
`qnwcheb`	$1/\sqrt{1-x^2}$	$[-1, 1]$	Chebyshev
`qnwnorm`	e^{-x^2} (Gaussian)	$(-\infty, \infty)$	Hermite
`qnwbeta`	$x^{a-1}(1-x)^{b-1}$	$[0, 1]$	Jacobi
`qnwgamma`	$x^{a-1}e^{-x}$	$[0, \infty)$	Laguerre

Why different rules? Each rule places nodes where the weight function $w(x)$ concentrates probability mass:

- **Gauss-Hermite** (`qnwnorm`): Nodes cluster around the mean. Optimal for normal distributions—the workhorse for computing expectations over income shocks.
- **Gauss-Legendre** (`qnwlege`): Nodes spread across the interval with slight concentration at endpoints. Best for smooth functions on bounded intervals.
- **Gauss-Chebyshev** (`qnwcheb`): Nodes are Chebyshev points (cosines of equally-spaced angles). Optimal for polynomial interpolation and avoids Runge's phenomenon.
- **Gauss-Jacobi** (`qnwbeta`): Handles Beta distributions on $[0, 1]$.
- **Gauss-Laguerre** (`qnwgamma`): For Gamma distributions with positive support.

Convenience wrappers. Some functions adapt the basic rules for common use cases:

- `qnwlogn`: Transforms Hermite nodes via exponentiation for lognormal distributions.
- `qnwunif`: Adapts Legendre and normalizes weights for uniform density.
- `qnwnorm_mv`: Tensor product of Hermite rules for multivariate normal.

Non-Gaussian rules. The functions `qnwsimp` (Simpson) and `qnwtrap` (Trapezoidal) are **Newton-Cotes** quadrature, which use equally-spaced nodes. These are simpler but less accurate for smooth functions. Use them when function values are only available on a pre-specified grid.

2.2.2 Function: `qnwlege`

```
def qnwlege(n: int, a: float, b: float) -> Tuple[np.ndarray, np.ndarray]
```

Purpose.
Gauss-Legendre quadrature for integrals over bounded intervals $[a, b]$. The weighting function is $w(x) = 1$ (uniform), so this approximates:

$$\int_a^b f(x)\,dx \approx \sum_{i=1}^{n} w_i f(x_i)$$

Arguments.

`n`
Number of nodes. With n nodes, polynomials of degree $\leq 2n - 1$ are integrated exactly.

`a, b`
Integration bounds.

Returns. Tuple `(nodes, weights)` for approximating $\int_a^b f(x)dx$.

When to use.
General-purpose integration on bounded intervals. Optimal for smooth functions.

2.2.3 Function: `qnwcheb`

```
def qnwcheb(n: int, a: float, b: float, kind: str = "gauss") -> Tuple[np.ndarray,
    np.ndarray]
```

Purpose.
Chebyshev-based quadrature on $[a, b]$. Two variants are supported:

- `kind="gauss"` (default): Gauss-Chebyshev with weighting function $w(x) = 1/\sqrt{1-x^2}$.
- `kind="clenshaw_curtis"`: Clenshaw–Curtis weights on Chebyshev nodes for unweighted integrals (CompEcon-compatible).

For the Gauss-Chebyshev case, this approximates:

$$\int_a^b \frac{f(x)}{\sqrt{1-x^2}}\,dx \approx \sum_{i=1}^{n} w_i\, f(x_i)$$

where x is scaled from $[a, b]$ to $[-1, 1]$.

Arguments.

`n`
Number of nodes (Chebyshev points).

`a, b`
Integration bounds (mapped to $[-1, 1]$).

`kind`
Either `"gauss"` or `"clenshaw_curtis"`.

Returns. Tuple `(nodes, weights)`.

When to use.

- When constructing Chebyshev polynomial approximations (nodes are optimal interpolation points).
- For integrals with weight function $1/\sqrt{1-x^2}$ (`kind="gauss"`).
- For unweighted integrals using Chebyshev nodes (`kind="clenshaw_curtis"`).
- When you want to avoid Runge's phenomenon in polynomial interpolation.

Special property: The Chebyshev nodes are $x_i = \cos\left(\frac{2i-1}{2n}\pi\right)$ for $i = 1, \ldots, n$, which are cosines of equally-spaced angles.

2.2.4 Function: `qnwnorm`

```
def qnwnorm(
    n: int,
    mu: float = 0.0,
    sigma: float = 1.0
) -> Tuple[np.ndarray, np.ndarray]
```

Purpose.
Gauss-Hermite quadrature for computing expectations under normal distributions. This is the most commonly used quadrature in macro for approximating:

$$\mathbb{E}[f(X)] \text{ where } X \sim \mathcal{N}(\mu, \sigma^2)$$

Arguments.

`n`
Number of quadrature nodes. Higher n = more accurate but more expensive. Typically 5-9 is sufficient.

`mu`
Mean of the normal distribution.

`sigma`
Standard deviation.

Returns. Tuple `(nodes, weights)` where `nodes` are evaluation points and `weights` sum to 1.

Connections. Used in `markov.py` for discretizing AR(1) processes; can be used directly for computing expectations in models with continuous shocks.

Example.

```
from hetmacro.quadrature import qnwnorm
import numpy as np

# Approximate E[exp(X)] where X ~ N(0, 0.1^2)
# True answer: exp(0.5 * 0.1^2) = 1.00501
```

```
nodes, weights = qnwnorm(n=5, mu=0.0, sigma=0.1)
approx = np.dot(weights, np.exp(nodes))
print(f"Approximation: {approx:.5f}")  # Very close to 1.00501
```

2.2.5 Function: qnwlogn

```
def qnwlogn(
    n: int,
    mu: float = 0.0,
    sigma: float = 1.0
) -> Tuple[np.ndarray, np.ndarray]
```

Purpose.
Quadrature for lognormal distributions. If $\log X \sim \mathcal{N}(\mu, \sigma^2)$, this provides nodes and weights for $\mathbb{E}[f(X)]$.

Note.
The nodes are exponentiated Gauss-Hermite nodes; weights remain the same.

2.2.6 Function: qnwunif

```
def qnwunif(n: int, a: float, b: float) -> Tuple[np.ndarray, np.ndarray]
```

Purpose.
Quadrature for uniform distribution on $[a, b]$. Weights are normalized so $\sum w_i = 1$.

2.2.7 Function: qnwbeta

```
def qnwbeta(n: int, a: float = 1.0, b: float = 1.0) -> Tuple[np.ndarray, np.ndarray
    ]
```

Purpose.
Quadrature for Beta(a, b) distribution on $[0, 1]$. Useful for modeling bounded random variables like labor supply elasticities or preference heterogeneity.

2.2.8 Function: `qnwgamma`

```
def qnwgamma(n: int, a: float = 1.0, b: float = 1.0) -> Tuple[np.ndarray, np.
    ndarray]
```

Purpose.
Quadrature for Gamma(a, b) distribution (shape a, scale b). Useful for modeling income distributions or shock processes with positive support.

2.2.9 Function: `qnwnorm_mv`

```
def qnwnorm_mv(
    n: np.ndarray,
    mu: np.ndarray,
    Sigma: np.ndarray
) -> Tuple[np.ndarray, np.ndarray]
```

Purpose.
Multivariate normal quadrature via tensor product. For $X \sim \mathcal{N}(\mu, \Sigma)$, computes nodes and weights for $\mathbb{E}[f(X)]$.

Arguments.

`n`
Array of number of nodes per dimension.

`mu`
Mean vector.

`Sigma`
Covariance matrix (must be positive definite).

Returns. Tuple `(nodes, weights)` where `nodes` has shape $(n_1 \times \cdots \times n_d, d)$.

Example.

```python
from hetmacro.quadrature import qnwnorm_mv
import numpy as np

# Bivariate normal with correlation
mu = np.array([0.0, 0.0])
Sigma = np.array([[1.0, 0.5], [0.5, 1.0]])
nodes, weights = qnwnorm_mv(n=np.array([5, 5]), mu=mu, Sigma=Sigma)
print(f"Number of quadrature points: {len(weights)}")  # 25
```

2.2.10 Functions: `qnwsimp`, `qnwtrap`

```python
def qnwsimp(n: int, a: float, b: float) -> Tuple[np.ndarray, np.ndarray]
def qnwtrap(n: int, a: float, b: float) -> Tuple[np.ndarray, np.ndarray]
```

Purpose.
Newton-Cotes rules (Simpson's and Trapezoidal) for numerical integration. These use equally-spaced nodes and are simpler but less accurate than Gaussian quadrature for smooth functions.

When to use.
When function values are only available on a pre-specified grid (e.g., integrating over a distribution that's already on an asset grid).

2.3 `interpolation.py` – Function Approximation

2.3.1 Introduction and Economic Motivation

In dynamic programming, we iterate on functions defined on grids. However, optimal policies often require evaluating functions at off-grid points. For example, in the Endogenous Grid Method (EGM), we compute:

$$c^*(a, e) = c_{\text{endog}}(a^{\text{endog}}(a, e))$$

where a^{endog} may not lie on the asset grid. Interpolation fills this gap.

Additionally, for computing stationary distributions with continuous policies, we need the "lottery" method: given a policy $a'(a, e)$ that falls between grid points $a_i < a' < a_{i+1}$, we split the mass probabilistically.

2.3.2 Function: `interp_linear`

```
def interp_linear(
    x_grid: np.ndarray,
    y: np.ndarray,
    x_new: np.ndarray
) -> np.ndarray
```

Purpose.
Vectorized 1D linear interpolation. Given function values $y = f(x_{\text{grid}})$, evaluate at new points x_{new}.

Arguments.

`x_grid`
1D array of grid points (must be sorted ascending).

`y`
Function values. Shape `(n,)` for single function or `(m, n)` for m functions on the same grid.

`x_new`
Points at which to interpolate.

Returns. Interpolated values, same shape as `x_new` (or `(m,) + x_new.shape` for multiple functions).

Connections. Core building block for EGM (`backward.py`). Also used in `get_lottery` for distribution iteration.

Example.

```python
from hetmacro.interpolation import interp_linear
import numpy as np

# Interpolate consumption policy at off-grid asset values
a_grid = np.linspace(0, 10, 100)
c_policy = 0.05 * a_grid + 1.0  # Example: linear consumption
a_new = np.array([2.5, 5.5, 7.3])
c_new = interp_linear(a_grid, c_policy, a_new)
```

2.3.3 Function: `interp_bilinear`

```python
def interp_bilinear(
    x_grid: np.ndarray,
    y_grid: np.ndarray,
    z: np.ndarray,
    x_new: np.ndarray,
    y_new: np.ndarray,
) -> np.ndarray
```

Purpose.
Bilinear interpolation on a 2D rectilinear grid. For functions of two state variables $f(x, y)$.

Arguments.

`x_grid, y_grid`
1D arrays defining the grid in each dimension.

`z`
Function values, shape `(len(y_grid), len(x_grid))`.

`x_new, y_new`
Points at which to interpolate.

Returns. Interpolated values at `(x_new, y_new)` pairs.

2.3.4 Function: `get_lottery`

```
def get_lottery(
    a_policy: np.ndarray,
    a_grid: np.ndarray
) -> Tuple[np.ndarray, np.ndarray]
```

Purpose.
Compute lottery indices and probabilities for distribution iteration. When agents choose $a' \in (a_i, a_{i+1})$, we assign probability π_i to a_i and $1 - \pi_i$ to a_{i+1}, where:

$$\pi_i = \frac{a_{i+1} - a'}{a_{i+1} - a_i}$$

Arguments.

`a_policy`
Policy function values (any shape).

`a_grid`
Asset grid (1D, sorted).

Returns. Tuple `(a_i, a_pi)` where `a_i` is the lower grid index and `a_pi` is the probability on `a_i`.

Connections. Essential input to `forward_iteration` in `forward.py`. The "lottery" method is how we iterate distributions forward when policies are continuous.

Example.

```
from hetmacro.interpolation import get_lottery
import numpy as np

a_grid = np.array([0, 1, 2, 3, 4])
a_policy = np.array([0.5, 1.7, 2.3])  # Off-grid choices

a_i, a_pi = get_lottery(a_policy, a_grid)
# a_i = [0, 1, 2] -- lower indices
# a_pi = [0.5, 0.3, 0.7] -- probabilities on lower point
```

2.3.5 Chebyshev Approximation Functions

```
def cheb_nodes(n: int, a: float, b: float) -> np.ndarray
def cheb_basis(x: np.ndarray, n: int, a: float, b: float) -> np.ndarray
def cheb_coef(f: Callable, n: int, a: float, b: float) -> np.ndarray
def cheb_eval(coef: np.ndarray, x: np.ndarray, a: float, b: float) -> np.ndarray
```

Purpose.
Chebyshev polynomial approximation provides highly accurate function representation with minimal oscillation (Runge phenomenon avoidance).

Workflow.

1. Generate Chebyshev nodes with `cheb_nodes`
2. Fit coefficients with `cheb_coef` given a function
3. Evaluate approximation at arbitrary points with `cheb_eval`

Example.

```
from hetmacro.interpolation import cheb_coef, cheb_eval
import numpy as np

# Approximate exp(x) on [-1, 1]
coef = cheb_coef(np.exp, n=10, a=-1, b=1)
x_test = np.linspace(-1, 1, 100)
approx = cheb_eval(coef, x_test, a=-1, b=1)
error = np.max(np.abs(approx - np.exp(x_test)))
print(f"Max error: {error:.2e}")  # Very small
```

Chebyshev Approximation in Practice (Problem Set 1)

For approximating nonlinear functions (e.g., value functions or Runge's function), compare accuracy across polynomial degrees. Using **Chebyshev nodes** avoids the Runge phenomenon that occurs with equidistant nodes.

Example: Runge's function. Approximate $f(x) = 1/(1+x^2)$ on $[-10, 10]$ (a classic test where equidistant polynomial interpolation diverges near the boundaries). With Chebyshev approximation, error decreases as n increases.

```
from hetmacro.interpolation import cheb_nodes, cheb_coef, cheb_eval
import numpy as np

def runge(x):
    return 1.0 / (1.0 + x**2)

a, b = -10.0, 10.0
x_fine = np.linspace(a, b, 500)
f_true = runge(x_fine)

# Compare polynomial degrees 5, 10, 15, 20
for n in [5, 10, 15, 20]:
    coef = cheb_coef(runge, n=n, a=a, b=b)
    approx = cheb_eval(coef, x_fine, a=a, b=b)
    err = np.max(np.abs(approx - f_true))
    print(f"n={n}: max error = {err:.2e}")
```

Visualizing approximation error. Plot `approx` and `f_true` on `x_fine` for each n to see how the error concentrates near the boundaries when n is small and decreases globally as n increases. For theoretical error bounds (Rivlin's theorem), see the Macro Bible Appendix B — Function Approximation.

2.3.6 Spline Functions

```
def spline_coef(x: np.ndarray, y: np.ndarray, kind: str = "cubic")
def spline_eval(coef, x_new: np.ndarray) -> np.ndarray
```

Purpose.
Cubic spline interpolation for smooth function approximation. Returns a SciPy spline object.

2.3.7 Basis Matrix Helpers (Spline Collocation)

```
def spline_basis_matrix(
    breakpoints: np.ndarray,
    x: np.ndarray,
    degree: int = 3
)
def cubic_basis_matrix(breakpoints: np.ndarray, x: np.ndarray)
def linear_basis_matrix(breakpoints: np.ndarray, x: np.ndarray)
def tensor_basis_matrix(Phi_a, Phi_z)
```

Purpose.

- `spline_basis_matrix`: sparse B-spline design matrix at evaluation points.
- `cubic_basis_matrix`: cubic basis ($k = 3$), used for value-function collocation.
- `linear_basis_matrix`: linear tent basis ($k = 1$), used for distribution transition matrices.
- `tensor_basis_matrix`: row-wise Kronecker product for 2D tensor-product collocation.

For n breakpoints and cubic splines with open knots, the number of basis functions is $n + 2$.

2.4 `optimize.py` – Root-Finding and Optimization

2.4.1 Introduction and Economic Motivation

Equilibrium in economic models often requires finding roots of excess demand functions or optimizing objective functions. For example:

- **Market clearing**: Find interest rate r^* such that $A(r^*) - K(r^*) = 0$.
- **Optimal policy**: Find consumption c^* maximizing $u(c) + \beta \mathbb{E}[V(a', e')]$.

This module wraps SciPy's optimization routines with a consistent API and sensible defaults.

Rigorous treatment. A macro-level, rigorous treatment of calibration (method of simulated moments), root finding (bisection, Brent, Newton, Broyden), optimization (golden section, Nelder–Mead, quasi-Newton), and Euler equation methods, with worked examples and figures from Problem Set 1, is given in the **Macro Bible** (Lecture Notes in Macroeconomics), Appendix B — Numerical Methods chapter (`Parts_and_chapters/Appendix/numeri`

2.4.2 Root-Finding Functions

`bisect`

```
def bisect(
    f: Callable,
    a: float,
    b: float,
    args=(),
    tol: float = 1e-12,
    max_iter: int = 100
) -> float
```

Purpose.
Bisection method for finding roots. Guaranteed to converge if $f(a)$ and $f(b)$ have opposite signs.

When to use.
For well-behaved monotonic functions like excess demand for capital. Simple and robust.

`brentq`

```
def brentq(
    f: Callable,
    a: float,
    b: float,
    args=(),
    tol: float = 1e-12
) -> float
```

Purpose.
Brent's method—combines bisection with inverse quadratic interpolation. Faster than bisection while retaining guaranteed convergence.

When to use.
Default choice for 1D root-finding with bracketed roots.

`newton`

```
def newton(
    f: Callable,
    x0: float,
    fprime: Callable,
    args=(),
    tol: float = 1e-8,
    max_iter: int = 50,
) -> float
```

Purpose.
Newton-Raphson method using derivative information. Quadratic convergence near root.

When to use.
When you have an analytical derivative and need fast convergence.

`secant`

```
def secant(
    f: Callable,
    x0: float,
    args=(),
    tol: float = 1e-8,
    max_iter: int = 50
) -> float
```

Purpose.
Secant method—derivative-free approximation of Newton's method.

`broyden` **(unified Broyden solver)**

```
def broyden(
    f: Callable,
    x0: np.ndarray,
    method: str = "compecon",
    *,
    args=(),
    tol=None,
    max_iter=None,
    a=None,
    b=None,
    return_info: bool = False,
    **kwargs,
)
```

Purpose.
Single entry point for Broyden-style solvers. Solves $F(x) = 0$ for systems of nonlinear equations. The argument `method` selects the implementation.

Arguments.

`method`
One of `"scipy"`, `"compecon"`, or `"componentwise"`.

`a, b`
Required when `method="componentwise"`: lower and upper bounds on x.

`**kwargs`
Passed to the underlying solver (e.g., `max_steps`, `show_iters` for `"compecon"`).

When to use which method:

- `method="compecon"`: Full coupled system $F(x) = 0$ (e.g., equilibrium residuals in all n unknowns). CompEcon-style: inverse Jacobian, backstepping line search, optional restarts. Use when the system is nonlinear or ill-scaled; SciPy's Broyden can diverge in such cases.
- `method="scipy"`: Same full-system case when the problem is well-behaved. Thin wrapper around `scipy.optimize.root(..., method="broyden1")`.

- `method="componentwise"`: Vectorized, largely *componentwise* equations with bounds $a \leq x \leq b$ (e.g., many independent 1D FOCs). Requires `a` and `b`. Use for the inner step of a fixed-point iteration (e.g., household FOCs for hours given wages).

Backward compatibility: `broyden_compecon(...)` and `solve_broyden(func, x, a, b, ...)` are thin wrappers that call `broyden` with the appropriate `method`.

2.4.3 Optimization Functions

`golden_search`

```
def golden_search(
    f: Callable,
    a: float,
    b: float,
    args=(),
    tol: float = 1e-8
) -> float
```

Purpose.
Golden section search for 1D minimization on a bounded interval.

When to use.
Minimizing unimodal functions when derivative is unavailable.

`brent_min`

```
def brent_min(
    f: Callable,
    a: float,
    b: float,
    args=(),
    tol: float = 1e-8
) -> float
```

Purpose.
Brent's method for 1D minimization. Generally faster than golden section.

`golden_max_vec`

```
def golden_max_vec(
    f: Callable,
    a: np.ndarray | float,
    b: np.ndarray | float,
    args=(),
    tol: float = 1e-8,
) -> tuple[np.ndarray, np.ndarray]
```

Purpose.
Vectorized golden-section *maximization* on per-element intervals. Accepts arrays `a`, `b` (bounds for each element) and a vectorized objective `f`; returns maximizers `x` and `f(x)`.

When to use.
Many independent 1D maximization problems (e.g. Bellman policy step over a grid) without Python loops.

`nelder_mead`

```
def nelder_mead(
    f: Callable,
    x0: np.ndarray,
    args=(),
    tol: float = 1e-8,
    callback=None,
    options: dict | None = None,
    return_info: bool = False
)
```

Purpose.
Nelder-Mead simplex algorithm for derivative-free multi-dimensional optimization (analogous to MATLAB's `fminsearch`).

Returns. If `return_info=False`: tuple `(x_opt, f_opt)`. If `return_info=True`: `(x_opt, f_opt, hist)` where `hist` has keys `"x"` (list of parameter vectors) and `"f"` (list of objective values at each iteration), so you can plot the error/SSE trace. Optional `callback(xk)`

is invoked each iteration; `options` are passed to `scipy.optimize.minimize` (e.g. `maxiter`, `disp`).

Example.

```
from hetmacro.optimize import bisect, brentq

# Find equilibrium interest rate
def excess_demand(r, A_supply_func, K_demand_func):
    return A_supply_func(r) - K_demand_func(r)

r_star = brentq(excess_demand, a=0.01, b=0.05,
                args=(A_supply, K_demand))
```

2.4.4 Choosing the Right Method

The table below summarizes when to use each root-finding and optimization method. The choice depends on dimensionality, derivative availability, and problem structure.

Root-Finding Methods

Method	Dimension	Requirements	Best For
`bisect`	1D	Bracketed root	Simple, guaranteed convergence
`brentq`	1D	Bracketed root	Default 1D choice; superlinear
`newton`	1D	Derivative $f'(x)$	Fast convergence near root
`secant`	1D	None	Derivative-free Newton approx.
`broyden`	nD	Initial guess	Systems of equations $F(x) = 0$

Optimization Methods

Method	Dimension	Requirements	Best For
`golden_search`	1D	Bounds $[a, b]$	Unimodal, no derivatives
`brent_min`	1D	Bounds $[a, b]$	Default 1D minimization
`nelder_mead`	nD	Initial guess	Derivative-free, noisy objectives

Decision Tree

1. **Root-finding or optimization?**

- Root-finding: solving $f(x) = 0$ (e.g., market clearing, Euler equations)
- Optimization: minimizing $f(x)$ (e.g., calibration, moment matching)

2. **One dimension or many?**

   - 1D root: use `brentq` (bracketed) or `newton` (with derivative)
   - 1D minimum: use `brent_min`
   - nD root: use `broyden` with `method="compecon"`
   - nD minimum: use `nelder_mead`

3. **Is the objective noisy or simulation-based?**

   - Yes: use `nelder_mead` (robust to noise, no gradients needed)
   - No: consider gradient-based methods if available

Example: Market Clearing vs Calibration

```
from hetmacro.optimize import brentq, nelder_mead

# Root-finding: find r* such that A(r) - K(r) = 0
r_star = brentq(lambda r: A(r) - K(r), a=0.01, b=0.05)

# Optimization: find params minimizing SSE of moments
def sse(params):
    model_moments = simulate_model(params)
    return np.sum((model_moments - target_moments)**2)

params_opt, sse_opt = nelder_mead(sse, x0=np.array([0.9, 0.1, 0.05]))
```

2.4.5 Calibration by Simulation (Method of Simulated Moments)

Many macroeconomic models require calibrating parameters to match empirical moments. When analytical moments are unavailable, we use **simulation-based calibration**: simulate the model, compute moments from simulated data, and minimize the distance to target moments.

Workflow

1. **Define target moments** from empirical data (e.g., standard deviation of log income, autocorrelations).

2. **Write a simulation function** that takes parameters and returns model-implied moments.

3. **Construct a loss function** measuring distance between model and target moments (typically sum of squared errors, SSE).

4. **Optimize** using `nelder_mead` (derivative-free, robust to simulation noise).

Example: Calibrating an Income Process

Consider calibrating an AR(1) process with measurement error:

$$\log y_t = \rho \log y_{t-1} + u_t, \quad u_t \sim \mathcal{N}(0, \sigma_u^2)$$

$$\log \tilde{y}_t = \log y_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma_\varepsilon^2)$$

Target moments: $\text{sd}(\log \tilde{y})$, $\text{sd}(\Delta \log \tilde{y})$, autocorrelations at lags 1, 3, 5.

```
import numpy as np
from hetmacro.optimize import nelder_mead

# Target moments from data
target = {
    'sd_logy': 0.92,
    'sd_dlogy': 0.35,
    'acf1': 0.89,
    'acf3': 0.78,
    'acf5': 0.71,
}

def simulate_income(rho, sigma_u, sigma_eps, T=10000, seed=42):
    """Simulate observed log income with measurement error."""
    np.random.seed(seed)
    logy = np.zeros(T)
    for t in range(1, T):
        logy[t] = rho * logy[t-1] + sigma_u * np.random.randn()
```

```
    logy_obs = logy + sigma_eps * np.random.randn(T)
    return logy_obs

def compute_moments(logy_obs):
    """Compute moments from simulated data."""
    dlogy = np.diff(logy_obs)
    return {
        'sd_logy': np.std(logy_obs),
        'sd_dlogy': np.std(dlogy),
        'acf1': np.corrcoef(logy_obs[1:], logy_obs[:-1])[0,1],
        'acf3': np.corrcoef(logy_obs[3:], logy_obs[:-3])[0,1],
        'acf5': np.corrcoef(logy_obs[5:], logy_obs[:-5])[0,1],
    }

def sse_loss(params):
    """Sum of squared errors between model and target moments."""
    rho, sigma_u, sigma_eps = params
    if rho <= 0 or rho >= 1 or sigma_u <= 0 or sigma_eps <= 0:
        return 1e10  # penalty for invalid params
    logy_obs = simulate_income(rho, sigma_u, sigma_eps)
    model = compute_moments(logy_obs)
    sse = sum((model[k] - target[k])**2 for k in target)
    return sse

# Initial guess and optimization
x0 = np.array([0.9, 0.2, 0.1])
params_opt, sse_opt, hist = nelder_mead(sse_loss, x0, return_info=True)

print(f"Optimal params: rho={params_opt[0]:.4f}, "
      f"sigma_u={params_opt[1]:.4f}, sigma_eps={params_opt[2]:.4f}")
print(f"Final SSE: {sse_opt:.6f}")
```

Convergence Diagnostics

When using `nelder_mead` with `return_info=True`, you receive a history dictionary for diagnostics:

```
import matplotlib.pyplot as plt

```

```
# Plot SSE trace
plt.figure(figsize=(8, 4))
plt.plot(hist['f'])
plt.xlabel('Iteration')
plt.ylabel('SSE')
plt.title('Nelder-Mead Convergence')
plt.yscale('log')
plt.show()
```

Best practices:

- **Multiple restarts**: Run from several initial guesses to avoid local minima.
- **Check final SSE**: A very small SSE suggests good fit; a large SSE may indicate model misspecification.
- **Examine moment fit**: Compare model and target moments individually to identify which are hardest to match.
- **Increase simulation length**: Longer simulations reduce Monte Carlo noise in moment estimates.

Appendix B (Macro Bible). For the MSM objective, weight matrix choice, stochastic process (AR(1)+noise and jump-diffusion), and figures from the income calibration problem set, see the Macro Bible (Lecture Notes in Macroeconomics), Appendix B — Numerical Methods, section "Calibration and Estimation".

2.5 `markov.py` – Markov Chain Tools

2.5.1 Introduction and Economic Motivation

Many economic models feature persistent idiosyncratic shocks, typically modeled as AR(1) processes:

$$\log e_{t+1} = \rho \log e_t + \sigma \varepsilon_{t+1}, \quad \varepsilon_t \sim \mathcal{N}(0,1)$$

For numerical solution, we discretize this continuous process into a finite-state Markov chain with states $\{e_1, \ldots, e_n\}$ and transition matrix Π where $\Pi_{ij} = \Pr(e_{t+1} = e_j \mid e_t = e_i)$.

Two main methods exist: **Tauchen** (older, based on CDF matching) and **Rouwenhorst** (more accurate for highly persistent processes).

2.5.2 Function: `rouwenhorst`

```
def rouwenhorst(
    n: int,
    rho: float,
    sigma: float,
    mu: float = 0.0
) -> Tuple[np.ndarray, np.ndarray]
```

Purpose.
Rouwenhorst discretization of AR(1) process. Preferred method for highly persistent processes ($\rho > 0.9$).

Arguments.

`n`
Number of states (typically 5-11).

`rho`
Persistence parameter of AR(1).

`sigma`
Standard deviation of innovation ε_t.

`mu`
Unconditional mean (default 0).

Returns. Tuple `(states, Pi)` where `states` is 1D array of state values and `Pi` is $n \times n$ transition matrix.

Connections. Output feeds directly into `backward.py` for computing conditional expectations in Bellman equations.

Example.

```
from hetmacro.markov import rouwenhorst, stationary_distribution

# Discretize typical income process
rho, sigma = 0.95, 0.1
```

```
e_states, Pi = rouwenhorst(n=7, rho=rho, sigma=sigma)

# Compute stationary distribution
pi = stationary_distribution(Pi)
print(f"States: {e_states}")
print(f"Stationary probabilities: {pi}")
```

2.5.3 Function: `tauchen`

```
def tauchen(
    n: int,
    rho: float,
    sigma: float,
    mu: float = 0.0,
    n_std: float = 3.0,
) -> Tuple[np.ndarray, np.ndarray]
```

Purpose.

Tauchen method for AR(1) discretization. Uses equally-spaced states covering $\pm n_{\text{std}}$ standard deviations.

Arguments.

`n`

Number of states.

`rho`

Persistence.

`sigma`

Innovation standard deviation.

`mu`

Unconditional mean.

`n_std`

Number of standard deviations to cover (default 3).

When to use.
For less persistent processes ($\rho < 0.9$); Rouwenhorst is generally preferred otherwise.

2.5.4 Function: `stationary_distribution`

```
def stationary_distribution(
    P: np.ndarray,
    method: str = "eigen",
    tol: float = 1e-12
) -> np.ndarray
```

Purpose.
Compute the stationary distribution π satisfying $\pi = \Pi'\pi$.

Arguments.

`P`
Transition matrix (rows sum to 1).

`method`
Either `"eigen"` (find unit eigenvalue) or `"iterate"` (power iteration).

`tol`
Convergence tolerance for iteration method.

Returns. 1D array of stationary probabilities summing to 1.

2.5.5 Function: `simulate_markov`

```
def simulate_markov(
    P: np.ndarray,
    s0: int,
    T: int,
    random_state=None
) -> np.ndarray
```

Purpose.
Simulate a sample path of Markov chain indices.

Arguments.

`P`

Transition matrix.

`s0`

Initial state index.

`T`

Number of periods to simulate.

`random_state`

Seed for reproducibility.

Returns. 1D integer array of state indices over time.

2.6 `utils.py` – Economic Primitives and Utilities

2.6.1 Introduction

This module provides commonly-used economic functions (CRRA utility, CES aggregator) and general numerical utilities (fixed-point iteration, numerical differentiation, timing).

2.6.2 Utility Functions

`crra_utility`

```
def crra_utility(c: np.ndarray, gamma: float) -> np.ndarray
```

Purpose.
Constant Relative Risk Aversion utility:

$$u(c) = \begin{cases} \frac{c^{1-\gamma}}{1-\gamma} & \gamma \neq 1 \\ \log(c) & \gamma = 1 \end{cases}$$

`crra_marginal`

```
def crra_marginal(c: np.ndarray, gamma: float) -> np.ndarray
```

Purpose.
Marginal utility $u'(c) = c^{-\gamma}$.

Connections. Used in EGM (`backward.py`) to recover consumption from marginal value.

`crra_inverse_marginal`

```
def crra_inverse_marginal(u: np.ndarray, gamma: float) -> np.ndarray
```

Purpose.
Inverse marginal utility $(u')^{-1}(m) = m^{-1/\gamma}$.

Connections. Core of EGM: given $u'(c) = \beta R \mathbb{E}[u'(c')]$, we invert to get c.

`ces_aggregator`

```
def ces_aggregator(
    x: np.ndarray,
    y: np.ndarray,
    sigma: float,
    alpha: float = 0.5
) -> np.ndarray
```

Purpose.
CES aggregator:

$$F(x, y) = (\alpha x^\rho + (1 - \alpha) y^\rho)^{1/\rho}, \quad \rho = \frac{\sigma - 1}{\sigma}$$

where σ is the elasticity of substitution. Special case $\sigma = 1$ gives Cobb-Douglas.

2.6.3 Numerical Utilities

`compute_fixed_point`

```
def compute_fixed_point(
    T: Callable,
    v0: np.ndarray,
    tol: float = 1e-6,
```

```
    max_iter: int = 1000,
    verbose: bool = False,
)
```

Purpose.
Generic fixed-point iteration $v = T(v)$. Iterates until $\|v_{n+1} - v_n\|_\infty < \text{tol}$.

Connections. Can be used for custom Bellman operators or any contraction mapping.

`numerical_derivative`

```
def numerical_derivative(
    f: Callable,
    x: float,
    h: float = 1e-7,
    method: str = "central"
) -> float
```

Purpose.
Numerical differentiation using forward, backward, or central differences.

`numerical_jacobian`

```
def numerical_jacobian(
    f: Callable,
    x: np.ndarray,
    h: float = 1e-7
) -> np.ndarray
```

Purpose.
Numerical Jacobian matrix of vector-valued function $f : \mathbb{R}^n \to \mathbb{R}^m$.

`tic, toc`

```
def tic()
def toc() -> float
```

Purpose.
MATLAB-style timing. Call `tic()` to start timer, `toc()` to get elapsed seconds.

2.7 Examples and Validation

2.7.1 `grids.py`

Purpose.
This section validates the grid constructors with small, visual tests. The examples are generated in the notebook: `hetmacro/notebooks/test_grids.ipynb`.
Notebook reference:

```
# Notebook used to generate figures
hetmacro/notebooks/test_grids.ipynb
```

Results: The figures below illustrate spacing behavior, concentration around kinks, and the curse-of-dimensionality comparison between tensor and Smolyak grids.

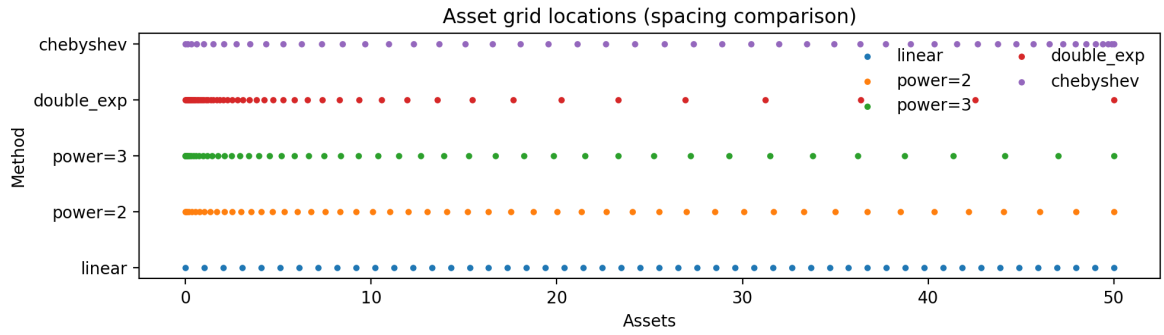


Figure 2.1: Asset grid spacing comparison: linear, power, double-exponential, and Chebyshev.

2.7.2 `quadrature.py`

Purpose.
This section validates the quadrature functions with visual tests showing node placement, weight distribution, and accuracy. The examples are generated in the notebook: `hetmacro/notebooks/test_quadrature.ipynb`.
Notebook reference:

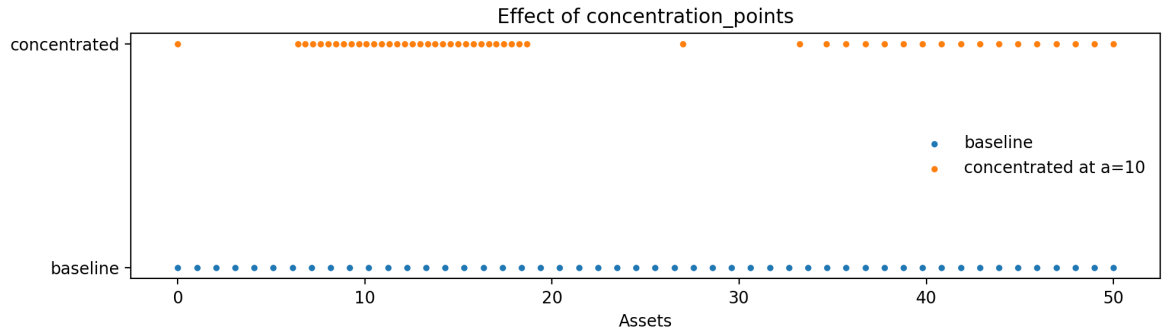


Figure 2.2: Concentrated grid points around a kink at $a = 10$ using `concentration_points`.

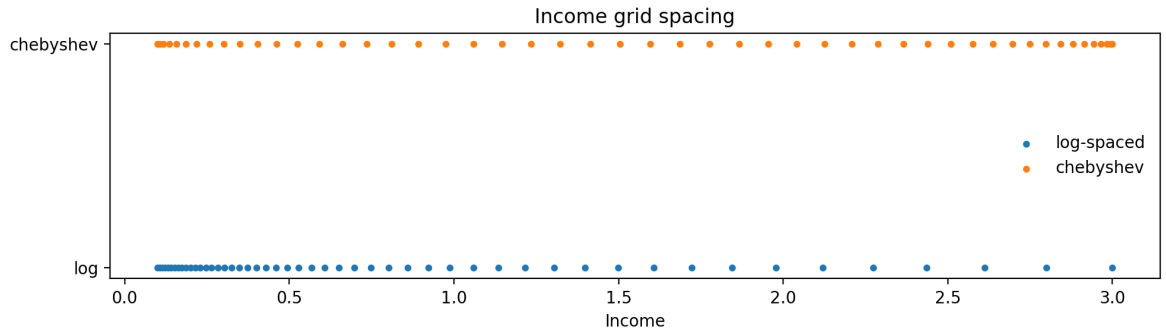


Figure 2.3: Income grid spacing: log-spaced vs Chebyshev nodes on a bounded interval.

```
# Notebook used to generate figures
hetmacro/notebooks/test_quadrature.ipynb
```

Results: Figure 2.5 displays all quadrature rules in a single panel. Each subplot shows nodes (circles, size proportional to weight) superimposed on the relevant function or PDF. The numerical tests in the notebook verify that all approximations match analytical values to high precision.

Subplot explanation:

Row 1 – Gaussian quadrature (bounded and unbounded intervals)

- **Gauss-Legendre:** Nodes for `qnwlege` on $[0, 1]$ integrating $f(x) = x^5$. Nodes spread across the interval with weights (circle sizes) roughly uniform—optimal for smooth functions on bounded domains.
- **Gauss-Chebyshev:** Nodes for `qnwcheb` shown with the weight function $w(x) = 1/\sqrt{1 - x^2}$. Nodes cluster near endpoints where the weight function diverges—these are the Chebyshev points, optimal for polynomial interpolation.

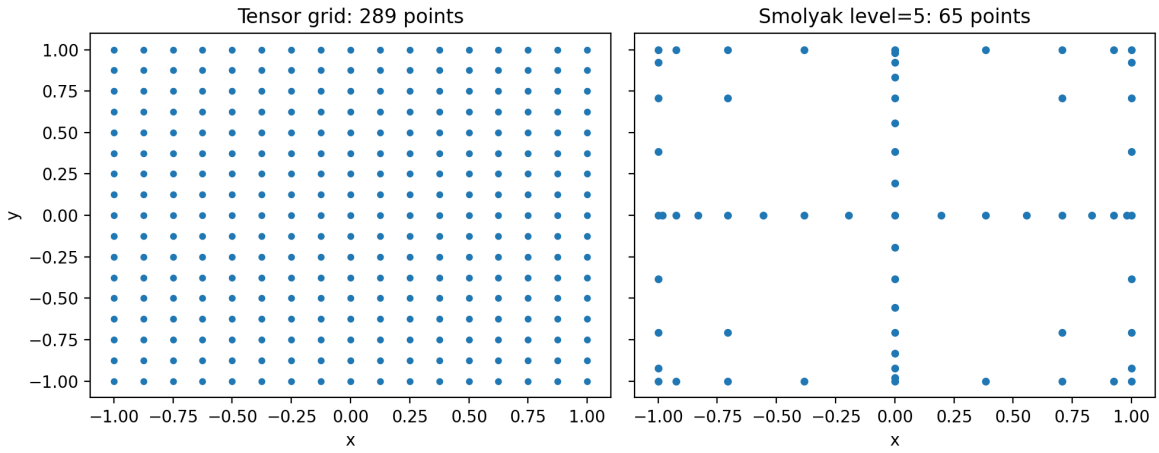


Figure 2.4: Tensor product vs Smolyak sparse grid (2D), highlighting point-count reduction.

- **Gauss-Hermite (Normal):** Nodes for `qnwnorm` overlaid on a normal PDF. Nodes cluster around the mean where probability mass concentrates, with larger weights (bigger circles) near the center.

Row 2 – Distribution-specific rules

- **Lognormal:** Nodes for `qnwlogn` on the lognormal PDF. These are exponentiated Hermite nodes, naturally clustering where the skewed distribution has most mass.
- **Uniform:** Nodes for `qnwunif` on $[0, 2]$. This is Legendre quadrature with weights normalized to sum to 1 (a probability distribution).
- **Simpson vs Trapezoid:** Newton-Cotes rules (`qnwsimp`, `qnwtrap`) for $\int_0^\pi \sin(x)\,dx = 2$. Both use equally-spaced nodes, but Simpson achieves higher accuracy through cubic interpolation.

Row 3 – Beta, Gamma, and multivariate normal

- **Beta:** Nodes for `qnwbeta` with Beta$(2, 5)$ PDF. Gauss-Jacobi nodes concentrate where the asymmetric Beta density peaks.
- **Gamma:** Nodes for `qnwgamma` with Gamma$(2, 3)$ PDF. Gauss-Laguerre nodes cover the positive support, clustering near the mode.
- **Multivariate Normal (2D):** Tensor-product nodes from `qnwnorm_mv` for a bivariate normal with correlation $\rho = 0.3$. The elliptical pattern reflects the covariance structure; larger circles indicate higher-weight nodes near the mean.

Row 4 – Curse of dimensionality

- **Node growth:** Total number of tensor-product nodes grows exponentially: n^d for n nodes per dimension in d dimensions. With $n = 5$, we have 5, 25, 125, 625, 3125 nodes for $d = 1, \ldots, 5$.

- **Error growth:** Despite exponentially more nodes, approximation error for $\mathbb{E}[\exp(\sum_i x_i)]$ under standard normal increases with dimension, illustrating the fundamental challenge of high-dimensional integration.

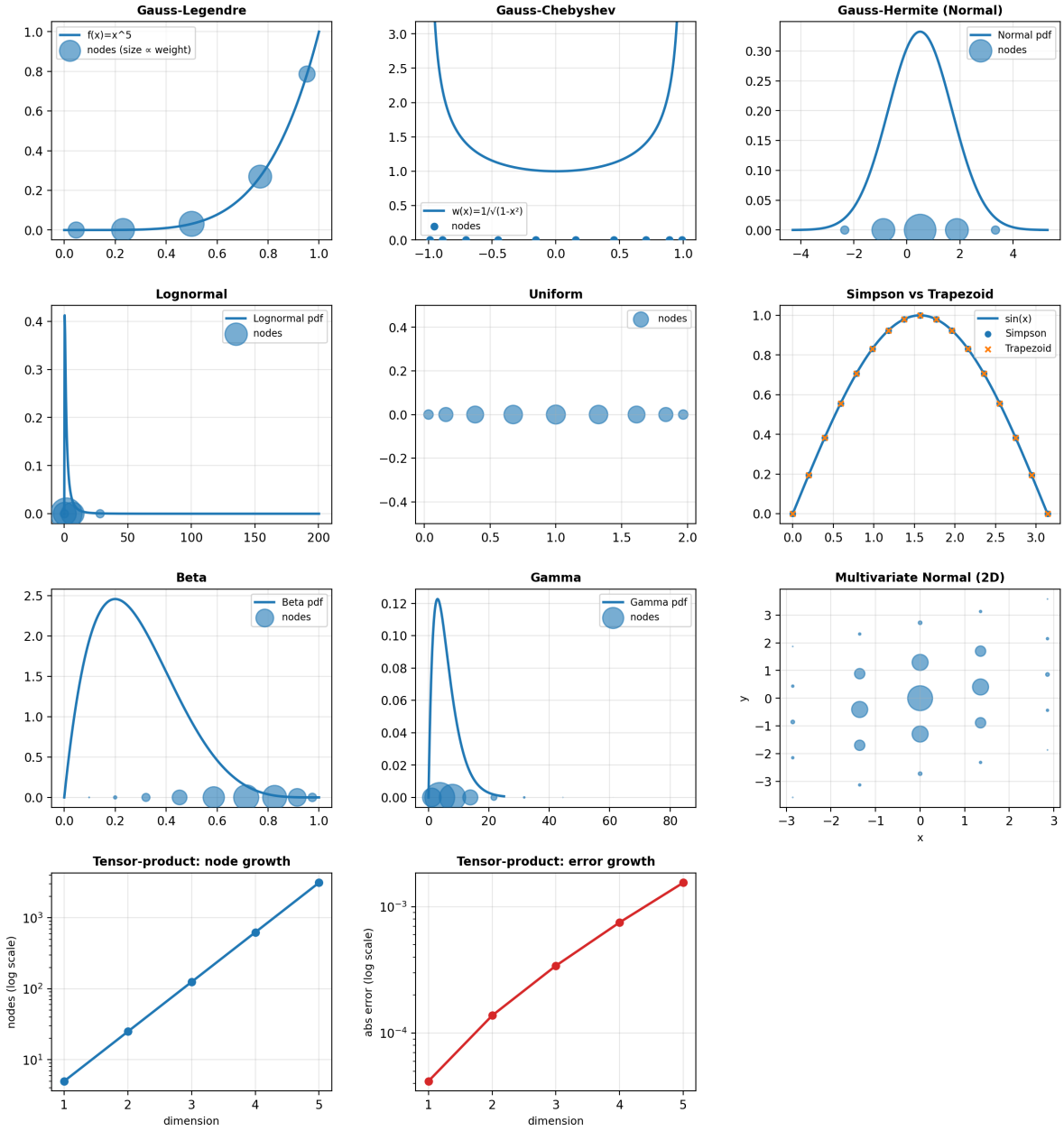


Figure 2.5: Quadrature rules: visual verification of node placement and weights.

CHAPTER 3

Heterogeneous Agent Tools

3.1 `backward.py` – Dynamic Programming Solvers

3.1.1 Introduction and Economic Motivation

The core of heterogeneous agent models is solving the household's Bellman equation:

$$V(a,e) = \max_{c,a'} \left\{ u(c) + \beta \mathbb{E}[V(a',e') \mid e] \right\}$$

subject to the budget constraint $c + a' = y(e) + (1+r)a$ and borrowing limit $a' \geq \underline{a}$.

Two main solution methods exist:

1. **Value Function Iteration (VFI)**: Iterate on V directly. Simple but slow.
2. **Endogenous Grid Method (EGM)**: Iterate on marginal value V_a. Much faster because it exploits the first-order condition to avoid numerical optimization.

3.1.2 Function: `backward_egm`

```
def backward_egm(
    Va: np.ndarray,
    Pi: np.ndarray,
    a_grid: np.ndarray,
```

```
    y: np.ndarray,
    r: float,
    beta: float,
    eis: float,
) -> Tuple[np.ndarray, np.ndarray, np.ndarray]
```

Purpose.
Single backward step using the Endogenous Grid Method. Given tomorrow's marginal value V'_a, compute today's policies.

Algorithm.

1. Compute expected marginal value: $W_a = \beta \Pi V'_a$
2. Invert FOC to get consumption on endogenous grid: $c_{\text{endog}} = W_a^{-\text{eis}}$
3. For each income state, interpolate to get policy on exogenous asset grid
4. Apply borrowing constraint; update marginal value

Arguments.

`Va`
Marginal value of assets, shape `(n_e, n_a)`.

`Pi`
Income transition matrix, shape `(n_e, n_e)`.

`a_grid`
Asset grid, shape `(n_a,)`.

`y`
Income levels for each state, shape `(n_e,)`.

`r`
Interest rate.

`beta`
Discount factor.

`eis`
Elasticity of intertemporal substitution ($= 1/\gamma$ for CRRA).

Returns. Tuple `(Va_new, a_policy, c_policy)` of updated marginal value and policy functions.

Connections. Called iteratively by `policy_iteration`. Policies feed into `forward.py` for distribution iteration.

Example.

```
from hetmacro.backward import backward_egm
import numpy as np

# Initial guess: consume everything
n_e, n_a = 7, 200
coh = y[:, np.newaxis] + (1 + r) * a_grid[np.newaxis, :]
Va_init = (1 + r) * (0.05 * coh) ** (-1/eis)

# One EGM step
Va_new, a_pol, c_pol = backward_egm(
    Va_init, Pi, a_grid, y, r=0.02, beta=0.96, eis=1.0
)
```

3.1.3 Function: `backward_vfi`

```
def backward_vfi(
    V: np.ndarray,
    Pi: np.ndarray,
    a_grid: np.ndarray,
    y: np.ndarray,
    r: float,
    beta: float,
    gamma: float,
) -> Tuple[np.ndarray, np.ndarray, np.ndarray]
```

Purpose.
Single VFI backward step with discrete choice over a' in the grid. Slower than EGM but more general (works with non-differentiable preferences).

Arguments.
Similar to `backward_egm`, but uses `gamma` (risk aversion) instead of `eis`.

Returns. Tuple `(V_new, a_policy, c_policy)`.

3.1.4 Function: `policy_iteration`

```
def policy_iteration(
    Pi: np.ndarray,
    a_grid: np.ndarray,
    y: np.ndarray,
    r: float,
    beta: float,
    eis: float,
    method: str = "egm",
    tol: float = 1e-9,
    max_iter: int = 10000,
) -> Tuple[np.ndarray, np.ndarray, np.ndarray]
```

Purpose.
Iterate backward steps until policy functions converge.

Arguments.

`method`
Either `"egm"` or `"vfi"`.

`tol`
Convergence tolerance on policy function.

`max_iter`
Maximum iterations.

Returns. Converged `(Va, a_policy, c_policy)`.

Connections. Main entry point for solving household problem. Output is used by `forward.py` and `steady_state.py`.

3.2 forward.py – Distribution Iteration

3.2.1 Introduction and Economic Motivation

Given optimal policies, we need the *distribution* of agents across states to compute aggregates:

$$A = \int a \, dD(a,e), \quad C = \int c(a,e) \, dD(a,e)$$

The stationary distribution D^* satisfies a fixed-point condition: if agents follow policy $a'(a,e)$ and income transitions according to Π, then D^* is invariant.

3.2.2 Function: forward_iteration

```
def forward_iteration(
    D: np.ndarray,
    Pi: np.ndarray,
    a_i: np.ndarray,
    a_pi: np.ndarray
) -> np.ndarray
```

Purpose.
Single forward step for distribution. Given current distribution D, compute next period's distribution after agents save according to policy and income transitions.

Arguments.

`D`
Current distribution, shape `(n_e, n_a)`.

`Pi`
Income transition matrix.

`a_i`
Lottery lower indices from `get_lottery`.

`a_pi`
Lottery probabilities from `get_lottery`.

Returns. Next period's distribution, same shape as `D`.

Connections. Uses output of `get_lottery` from `interpolation.py`.

3.2.3 Function: `stationary_distribution`

```
def stationary_distribution(
    Pi: np.ndarray,
    a_policy: np.ndarray,
    a_grid: np.ndarray,
    tol: float = 1e-10,
    max_iter: int = 10000,
) -> np.ndarray
```

Purpose.
Compute stationary distribution by iterating `forward_iteration` until convergence.

Arguments.

`Pi`
Income transition matrix.

`a_policy`
Asset policy function, shape `(n_e, n_a)`.

`a_grid`
Asset grid.

`tol`
Convergence tolerance.

`max_iter`
Maximum iterations.

Returns. Stationary distribution D^*, shape `(n_e, n_a)`, summing to 1.

Connections. Called by `household_steady_state` in `steady_state.py`.

3.2.4 Functions: `expectation_iteration`, `expectation_functions`

```
def expectation_iteration(X, Pi, a_i, a_pi) -> np.ndarray
def expectation_functions(X, Pi, a_i, a_pi, T) -> np.ndarray
```

Purpose.
Backward expectation operators for sequence-space Jacobian computation. Used internally by `ssj.py`.

3.2.5 Function: `stationary_eigenvector`

```
def stationary_eigenvector(Q, tol: float = 1e-12) -> np.ndarray
```

Purpose.
Recover stationary distribution as the left unit-eigenvector of a transition matrix:

$$n'Q = n', \quad \sum_i n_i = 1.$$

Uses sparse eigensolver for sparse matrices and dense eigendecomposition otherwise.

3.3 `steady_state.py` – Equilibrium Solvers

3.3.1 Introduction

Heterogeneous agent models require market clearing. In the Aiyagari model:

$$\underbrace{\int a \, dD(a, e; r)}_{\text{Asset supply } A(r)} = \underbrace{K(r)}_{\text{Capital demand}}$$

We solve for the equilibrium interest rate r^* that clears the asset market.

3.3.2 Function: `household_steady_state`

```
def household_steady_state(
    Pi: np.ndarray,
    a_grid: np.ndarray,
```

```
    y: np.ndarray,
    r: float,
    beta: float,
    eis: float,
    method: str = "egm",
    tol: float = 1e-9,
) -> Dict[str, np.ndarray]
```

Purpose.
Solve for household policies and distribution *given* prices. This is the "partial equilibrium" household block.

Arguments.

`Pi, a_grid, y, r, beta, eis`
Model parameters.

`method`
Solution method (`"egm"` or `"vfi"`).

`tol`
Convergence tolerance.

Returns. Dictionary containing:

- `"D"`: Stationary distribution
- `"Va"`: Marginal value function
- `"a"`, `"c"`: Policy functions
- `"a_i"`, `"a_pi"`: Lottery indices/weights
- `"A"`, `"C"`: Aggregate asset holdings and consumption
- All input parameters

Connections. Called repeatedly by `market_clearing` at different interest rates.

3.3.3 Function: market_clearing

```
def market_clearing(
    r_bounds: Tuple[float, float],
    household_ss_func: Callable[[float], Dict],
    K_demand_func: Callable[[float], float],
    tol: float = 1e-6,
    max_iter: int = 100,
) -> Dict[str, np.ndarray]
```

Purpose.
Find equilibrium interest rate using bisection on excess demand.

Arguments.

`r_bounds`
Tuple `(r_low, r_high)` bracketing equilibrium rate.

`household_ss_func`
Function mapping $r \mapsto$ household steady state dict.

`K_demand_func`
Function mapping $r \mapsto$ capital demand.

`tol`
Tolerance on excess demand.

`max_iter`
Maximum bisection iterations.

Returns. Household steady state dict at equilibrium r^*.

Example.

```
from hetmacro.steady_state import market_clearing

def hh_ss(r):
    y = w * np.exp(e)  # wage * productivity
    return household_steady_state(Pi, a_grid, y, r, beta, eis)

```

```
def K_demand(r):
    return (alpha / (r + delta)) ** (1/(1-alpha))

ss = market_clearing(
    r_bounds=(0.005, 0.04),
    household_ss_func=hh_ss,
    K_demand_func=K_demand
)
print(f"Equilibrium r = {ss['r']:.4f}")
```

3.4 `ssj.py` – Sequence-Space Jacobians

3.4.1 Introduction and Economic Motivation

For aggregate dynamics (impulse responses, business cycle analysis), we need to know how aggregates respond to shocks. The **sequence-space Jacobian** $\mathcal{J}$ captures:

$$\begin{pmatrix} dA_0 \\ dA_1 \\ \vdots \\ dA_{T-1} \end{pmatrix} = \mathcal{J} \begin{pmatrix} dr_0 \\ dr_1 \\ \vdots \\ dr_{T-1} \end{pmatrix}$$

The "fake news algorithm" (Auclert et al., 2021) efficiently computes these Jacobians by exploiting the structure of the household problem.

3.4.2 Function: `jacobian`

```
def jacobian(
    ss: Dict[str, np.ndarray],
    shocks: Dict[str, Dict[str, float]],
    T: int
) -> Dict[str, Dict[str, np.ndarray]]
```

Purpose.
Compute Jacobians of aggregate outputs (A, C) with respect to input shocks.

Arguments.

`ss`
Steady state dict from `household_steady_state`.

`shocks`
Dict mapping shock names to input perturbations. E.g., `{"r": {"r": 1.0}}` for interest rate shock.

`T`
Horizon length.

Returns. Nested dict: `Js["A"]["r"]` is the $T \times T$ Jacobian of assets w.r.t. interest rate.

Example.

```
from hetmacro.ssj import jacobian

# Compute Jacobian at steady state
shocks = {"r": {"r": 1.0}}  # unit shock to r
T = 300
Js = jacobian(ss, shocks, T)

# Impulse response of assets to 1% interest rate shock
dA = Js["A"]["r"] @ (0.01 * np.ones(T))
```

3.4.3 Function: `step1_backward`

```
def step1_backward(
    ss: Dict,
    shock: Dict[str, float],
    T: int,
    h: float = 1e-4,
) -> Tuple[Dict[str, np.ndarray], np.ndarray]
```

Purpose.
Step 1 of fake news algorithm: compute "curlyY" (direct effect) and "curlyD" (distribution response).

3.4.4 Function: `J_from_F`

```
def J_from_F(F: np.ndarray) -> np.ndarray
```

Purpose.
Convert fake news matrix F to Jacobian J via cumulative summation.

CHAPTER 4

Example: Aiyagari Model

4.1 `models/aiyagari.py`

This module demonstrates how to combine `hetmacro` tools to solve a complete model.

4.1.1 Function: `solve_aiyagari_ge`

```
def solve_aiyagari_ge(
    solver,
    income_process: IncomeProcess,
    a_grid: np.ndarray,
    beta: float,
    gamma: float,
    alpha: float = 0.36,
    delta: float = 0.08,
    r_bounds: Tuple[float, float] = (0.005, 0.04),
    tol_ge: float = 1e-6,
    method: str = "brentq",
    max_iter: int = 100,
    verbose: bool = False,
    solver_kwargs: Optional[Dict] = None,
) -> Dict[str, Any]
```

Purpose.
Solve the Aiyagari model GE steady state with any solver backend.

Workflow.

1. Accept a solver object (EGM, GridVFI, PFI, HowardGrid, etc.)
2. For each candidate r: compute prices, build `Household`, solve, compute ergodic distribution and aggregates
3. Find market-clearing r^* using Brent (`method="brentq"`) or bisection (`method="bisection"`)

Returns: Dictionary with `r, K, w, Y, C, household, solved_policy, distribution, aggregates, solver_name, method, n_iterations, elapsed`.

4.1.2 Function: `capital_supply_curve`

Evaluates household capital supply $K_s(r)$ on a grid of interest rates for plotting supply vs demand curves.

Example.

```
from hetmacro.models.aiyagari import solve_aiyagari_ge, capital_supply_curve
from hetmacro.income_process import RouwenhorstIncome
from hetmacro.grids import make_asset_grid
from hetmacro.solvers.egm import EGM

income = RouwenhorstIncome.from_ar1(n=7, rho=0.9, sigma=0.2)
a_grid = make_asset_grid(0, 50, 500, curvature=2.0)

# Solve GE
ss = solve_aiyagari_ge(
    EGM(), income, a_grid,
    beta=0.96, gamma=1.0, verbose=True
)
print(f"r* = {ss['r']:.4f}, K* = {ss['K']:.2f}")

# Supply/demand curves
curves = capital_supply_curve(
    EGM(), income, a_grid,
```

```
    beta=0.96, gamma=1.0, verbose=True
)
```

CHAPTER 5

Worked Examples from Problem Sets

This chapter summarizes how the tools in `hetmacro` combine to solve the main numerical tasks in Problem Set 1: root finding for equilibrium, calibration by simulation, dynamic programming, and Euler equation methods. Each section gives the economic setup, the relevant `hetmacro` functions, and a compact workflow. For full theory, see the Macro Bible (Lecture Notes in Macroeconomics), Appendix B — Numerical Methods.

5.1 Root Finding for Market Clearing (Problem 2)

Setup. Static labor supply equilibrium with heterogeneous ability e: households choose hours $h(e)$; production $Y = L^{\eta}$; wage $W = \eta L^{\eta-1}$; government rebates tax revenue lump-sum. Equilibrium is a vector $\mathbf{h}$ such that each household's FOC holds and the labor market clears. This reduces to solving $F(\mathbf{h}) = \mathbf{0}$ where F_i is the residual of the i-th household's first-order condition.

Method. Use `broyden` with `method='compecon'` for the full coupled system. Alternative: fixed-point iteration on the wage W with an inner `broyden(..., method='componentwise', a=0, b=hmax)` for the vector of hours given W.

Code pattern.

```
from hetmacro.optimize import broyden
import numpy as np
```

```

def residual(h, omega, e, eta, tau, sigma, gamma):
    """FOC residuals: h_i^gamma - (1-tau)*W*e_i*c_i^{-sigma}."""
    L = np.dot(omega, e * h)
    W = eta * L**(eta - 1)
    T = L**eta - (1 - tau) * W * L
    c = T + (1 - tau) * W * e * h
    return h**gamma - (1 - tau) * W * e * (c**(-sigma))

h0 = np.full(n, 0.3)  # initial guess
h_star, info = broyden(lambda h: residual(h, omega, e, eta, tau, sigma, gamma),
                       h0, method='compecon', tol=1e-10)
```

Convergence. Plot $\|F(\mathbf{h}^{(k)})\|$ vs. iteration to verify convergence. See codebook `optimize.py` and Macro Bible Appendix B for Broyden theory and labor market derivation.

5.2 Calibration by Simulation (Problem 3)

Setup. Calibrate an AR(1) process with measurement error for log productivity: $y_t = x_t + \varepsilon_t$, $x_t = \rho x_{t-1} + u_t$. Target moments: sd($\log y$), sd($\Delta \log y$), autocorrelations at lags 1, 3, 5.

Method. Simulate a panel, compute model moments, minimize sum of squared errors (SSE) between model and target moments using `nelder_mead`. Extension: jump-diffusion for fat tails (see Calibration Workflows in `macro_agents.md`).

Code pattern.

```
from hetmacro.optimize import nelder_mead
import numpy as np

def sse_loss(params):
    rho, sigma_u, sigma_eps = params
    if not (0 < rho < 1 and sigma_u > 0 and sigma_eps > 0):
        return 1e10
    logy = simulate_ar1_noise(rho, sigma_u, sigma_eps, T=10000)
    model = compute_moments(logy)  # sd, acf1, acf3, acf5
    return np.sum((np.array([model['sd'], model['acf1'], ...]) - target)**2)
```

```

params_opt, sse_opt, hist = nelder_mead(sse_loss, x0=np.array([0.9, 0.2, 0.1]),
                                        return_info=True)
# Plot hist['f'] for SSE convergence
```

See `optimize.py` "Calibration by Simulation" and Macro Bible "Calibration and Estimation" for MSM and jump-diffusion formulas.

5.3 Dynamic Programming (Problem 4)

Setup. Two-period or multi-period consumption-savings with income risk. Solve for value function $V(a, e)$ and policy $a'(a, e)$ (and consumption $c(a, e)$) via Bellman iteration.

Method. Use `backward.policy_iteration` with `method='vfi'` (value function iteration) or `method='egm'` (endogenous grid). Grids from `grids.py`; income process from `markov.rouwenhorst`; expectations via `quadrature` or the Markov chain.

Code pattern.

```
from hetmacro.grids import make_grid_1d
from hetmacro.markov import rouwenhorst
from hetmacro.backward import policy_iteration

a_grid = make_grid_1d(0, 50, 200, spacing='power', power=2.0)
e, Pi = rouwenhorst(n=7, rho=0.9, sigma=0.2)
y = np.exp(e)
Va, a_pol, c_pol = policy_iteration(Pi, a_grid, y, r=0.04, beta=0.96,
                                    eis=0.5, method='vfi')
```

Policy functions are on the grid; use `interp_linear` to evaluate at off-grid points. For projection/collocation methods, keep **solve grid** and **output/evaluation grid** separate: solve on basis nodes (Greville/Chebyshev), then evaluate policies on a common fine grid before distribution iteration and method comparison.

5.4 Euler Equation Methods (Problem 5)

Setup. Portfolio choice or consumption-savings where the solution is characterized by Euler equations. At an interior optimum, $u'(c_t) = \beta(1 + r)\mathbb{E}[u'(c_{t+1})]$ (and similarly for

risky asset). Solve by time iteration: guess tomorrow's policy, solve today's from the Euler equation, repeat.

Method. For each state (a, e) on the grid, solve the nonlinear equation (Euler residual = 0) for today's choice; use `broyden` or `brentq` for the inner 1D solve. Quadrature (`quadrature.qnwnorm` or `qnwnorm_mv`) for expectations. Check accuracy with the **Euler residual**: left-hand side minus right-hand side of the Euler equation; should be small in the interior.

Code pattern.

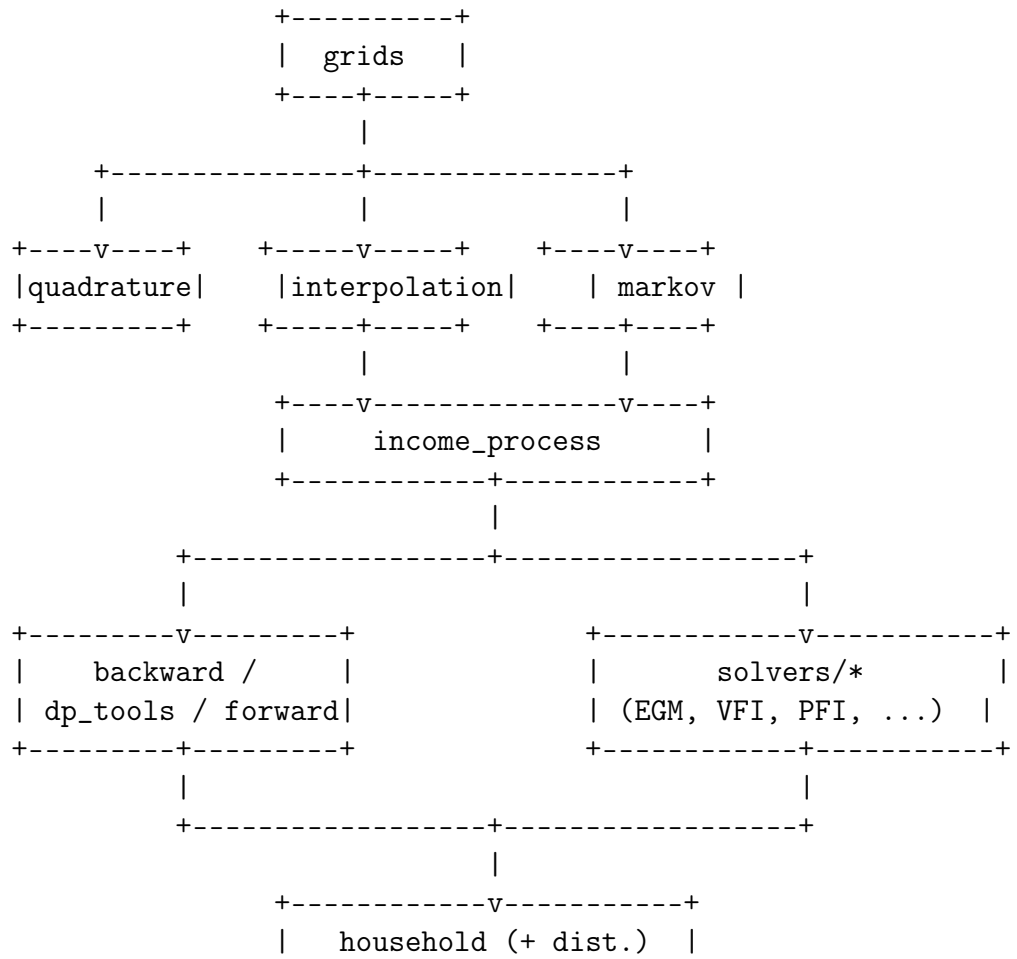
```
from hetmacro.quadrature import qnwnorm_mv
from hetmacro.optimize import broyden  # or brentq for 1D

# For each (a, e): find x, x_s that satisfy Euler equations
# res_safe = 1 - beta * E[M2 * (1+r)]
# res_risky = 1 - beta * E[M2 * (1+r+eps)]
nodes, weights = qnwnorm_mv(n=[7, 7], mu=mu, Sigma=Sigma)
# ... define residual function, then broyden(residual, x0)
```

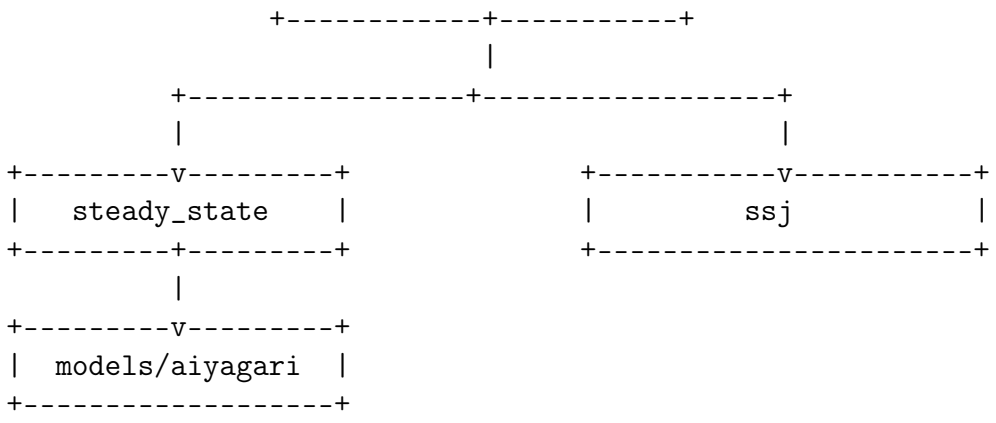
Plot the Euler residual over the state space to verify accuracy. See Macro Bible "Euler Equation Methods" and "Euler Residual Diagnostics".

CHAPTER 6

Quick Reference

6.1 Module Dependency Graph





6.2 Common Workflows

6.3 Public API Sync Checklist

This checklist is the compact, module-level API reference for agents and users. The rule is simple: if a public function in `hetmacro/*.py` changes, update this section and any relevant narrative sections before rebuilding `codebook.pdf`.

6.3.1 Foundational Modules

- `grids.py`: `make_asset_grid`, `power_grid`, `make_grid`, `double_exponential_grid`, `log_spaced_grid`, `make_grid_1d`, `make_grid_nd`, `cartesian_product`, `gridmake`, `tensor_product`, `smolyak_sparse_grid`

- `quadrature.py`: `qnwlege`, `qnwcheb`, `qnwnorm` (`sigma` or `sig2`), `qnwlogn`, `qnwunif`, `qnwsimp`, `qnwtrap`, `qnwbeta`, `qnwgamma`, `qnwnorm_mv`

- `interpolation.py`: `FunctionSpace`, `interp_linear`, `interp_bilinear`, `get_lottery`, `spline_fit`, `spline_coef`, `spline_eval`, `spline_basis_matrix`, `cubic_basis_matrix`, `linear_basis_matrix`, `tensor_basis_matrix`, `cheb_nodes`, `cheb_basis`, `cheb_coef`, `cheb_eval`

- `optimize.py`: `bisect`, `brentq`, `newton`, `secant`, `broyden`, `broyden_compecon`, `solve_broyden`, `golden_search`, `golden_max_vec`, `brent_min`, `nelder_mead`, `minimize_lbfgsb`

- `markov.py`: `rouwenhorst`, `tauchen`, `stationary_distribution`, `simulate_markov`, `simulate_mc`, `markov_chain_ar1`

- `utils.py`: `crra_utility`, `crra_marginal`, `crra_inverse_marginal`, `ces_aggregator`, `compute_fixed_point`, `numerical_derivative`, `numerical_jacobian`, `tic`, `toc`

6.3.2 Heterogeneous-Agent Modules

- backward.py: backward_egm, backward_vfi, policy_iteration, policy_evaluation_discrete, howard_policy_evaluation, coefficient_vfi_step
- forward.py: forward_iteration, stationary_distribution, expectation_iteration, expectation_functions, stationary_markov, stationary_eigenvector, update_policy_1a, update_exogenous_1a
- steady_state.py: household_steady_state, market_clearing
- distributions.py: forward_policy_step, stationary_distribution_ha, lottery_indices, compute_joint_transition_matrix, stationary_dist
- dp_tools.py: solve_policy_egm, solve_policy_vfi, solve_steady_policy, interp_grid, solve_euler_one_asset
- income_process.py: DiscreteIncome, RouwenhorstIncome, TauchenIncome, ContinuousQuadratu
- household.py: SolvedPolicy, Household
- solvers/*: GridVFI, HowardGrid, HowardImprovement, PolicyFunctionIteration, EGM, CollocationVFI_Spline, CollocationVFI_Chebyshev, NaiveEulerIteration, HowardEulerIteration
- ssj.py: jacobian, step1_backward, J_from_F
- models/aiyagari.py: firm_k_demand, firm_wage, capital_supply_curve, solve_aiyagari_ge

Use this section as a checklist during releases and after API edits.

6.3.3 Solving a Steady State

```
from hetmacro.grids import make_asset_grid
from hetmacro.markov import rouwenhorst
from hetmacro.steady_state import household_steady_state, market_clearing

# 1. Set up grids and income process
a_grid = make_asset_grid(0, 50, 500)
e, Pi = rouwenhorst(7, rho=0.95, sigma=0.1)

# 2. Define household problem
def hh_ss(r):
    y = w(r) * np.exp(e)
    return household_steady_state(Pi, a_grid, y, r, beta, eis)

# 3. Find equilibrium
ss = market_clearing((0.01, 0.04), hh_ss, K_demand)
```

6.3.4 Computing Impulse Responses

```
from hetmacro.ssj import jacobian

# 1. Get steady state
ss = solve_aiyagari_ge(...)

# 2. Compute Jacobians
Js = jacobian(ss, {"r": {"r": 1.0}}, T=300)

# 3. Construct impulse response
shock_path = np.zeros(300)
shock_path[0] = 0.01  # 1% shock at t=0
dA = Js["A"]["r"] @ shock_path
```